

Aether — Observability & Monitoring

Built-in Metrics, Events, and Health Tracking

Prometheus-native metrics, real-time health monitoring, audit trails with integrity hashes, and multi-cloud cost estimation — all without external dependencies.

Zero-Config Observability

Prometheus Metrics

Aether exposes a /metrics endpoint in Prometheus format. Scrape it directly or use the built-in dashboard.



Export Metrics

Start the metrics server on the default port and point Prometheus at it.

```
$ aether metrics --port 9090

Metrics server listening on :9090
Endpoint: /metrics
Format:   Prometheus text/plain

# Scrape config (prometheus.yml)
scrape_configs:
  - job_name: 'aether'
    static_configs:
      - targets: ['localhost:9090']
```



Key Metric Families

Core metrics exposed by every Aether instance.

```
# Workload metrics
aether_workloads_total{runtime="podman"} 5
aether_workloads_running{runtime="k8s"} 3
aether_workloads_failed 0

# Command duration (histogram)
aether_command_duration_seconds_bucket{
  cmd="run", le="1.0"} 42
aether_command_duration_seconds_sum{
  cmd="run"} 38.7

# Migration metrics
aether_migrations_total{strategy="blue-green"} 7
aether_migration_duration_seconds 34.2
```

Pre-built Grafana Dashboard (14 Panels)

Workload Overview

- Total workloads by runtime
- Running vs failed ratio
- Replica health status
- Deployment frequency

Performance

- Command duration (p50/p95/p99)
- Build time trends
- Migration latency
- API response times

Resources & Costs

- CPU/memory utilization
- Network I/O per workload
- Disk usage trends
- Estimated cost by provider

```
$ aether metrics dashboard --export grafana > aether-dashboard.json
# Import aether-dashboard.json into Grafana
# Dashboard ID: aether-overview | 14 panels | auto-refresh: 15s
```

14

Grafana Panels

50+

Metric Series

15s

Scrape Interval

0

External Deps

Health Monitoring

Continuous health tracking with history, circuit breaker protection, and automated recovery.



Health Status

```
$ aether health
WORKLOAD    STATUS    UPTIME    CHECKS    LAST
my-web-app   Healthy   99.97%    1,482     2s ago
api-service  Healthy   99.91%    1,201     5s ago
cache-redis  Healthy   100.0%    1,482     1s ago
worker-queue Warning   98.12%    890       3s ago

Overall: 3 healthy, 1 warning, 0 critical
```



Health History Timeline

```
$ aether health my-web-app --history 24h

Timeline (last 24h):
00:00-06:00 ██████████ Healthy
06:00-06:05 ████ Warning (high latency)
06:05-12:00 ██████████ Healthy
12:00-18:00 ██████████ Healthy
18:00-00:00 ██████████ Healthy

Uptime: 99.97% | Incidents: 1 | MTTR: 5m
```

Active Health Checks

Custom Health Check

```
$ aether health-check my-web-app \
  --type http \
  --path /healthz \
  --interval 10s \
  --timeout 3s \
  --threshold 3

Health check configured:
Type:      HTTP GET
Endpoint:   http://my-web-app:80/healthz
Interval:   every 10s
Timeout:    3s
Fail threshold: 3 consecutive
```

Circuit Breaker Protection

Aether automatically applies circuit breaker logic to prevent cascading failures.

Circuit Breaker States

- **Closed** — Normal operation, requests pass through
- **Open** — Failures exceeded threshold, requests blocked for cooldown
- **Half-Open** — Probe requests sent to test recovery

```
$ aether health my-web-app --circuit-breaker
State: CLOSED
Failures: 0/3 threshold
Success rate: 100% (last 100 checks)
Last trip: 6d ago (auto-recovered)
```

Events & Audit Trail

Every action is recorded. Every event is verifiable. Every notification is delivered.

Event System with Severity Levels

Event Stream

```
$ aether events --follow
```

TIMESTAMP	SEVERITY	SOURCE	MESSAGE
2026-04-12T10:00:01	INFO	scheduler	Workload my-web-app scaled to 3
2026-04-12T10:00:05	INFO	health	Health check passed (12ms)
2026-04-12T10:01:12	WARNING	resources	CPU usage at 85% threshold
2026-04-12T10:02:30	INFO	migration	Migration plan created
2026-04-12T10:02:45	INFO	migration	Blue-green switch complete
2026-04-12T10:05:00	CRITICAL	health	Endpoint /api unresponsive

Severity Levels

Level	Description	Action
INFO	Normal operations	Log only
WARNING	Threshold approaching	Log + alert
CRITICAL	Service degraded	Log + alert + auto-heal
FATAL	Complete failure	Log + alert + page

Audit Trail with Integrity Hashes

```
$ aether audit --last 10
```

ID	TIMESTAMP	USER	ACTION	TARGET	HASH (SHA-256)
a8f301	2026-04-12T10:00:01	admin	deploy	my-web-app	e3b0c44298fc1c14...
a8f302	2026-04-12T10:02:30	admin	migrate.plan	my-web-app	9f86d081884c7d65...
a8f303	2026-04-12T10:02:45	system	migrate.exec	my-web-app	a665a459204f70b8...
a8f304	2026-04-12T10:05:12	admin	scale	api-service	2cf24dba5fb0a30e...

```
# Verify audit chain integrity
$ aether audit --verify
Chain integrity: VALID (304 entries, 0 tampered)
```

Webhook Notifications

Configure Webhooks

```
$ aether notify add \
  --name ops-slack \
  --type webhook \
  --url https://hooks.slack.com/T0/B0/xxx \
  --events "critical,fatal" \
  --retry-count 3 \
  --retry-delay 10s
```

Webhook registered:

Name: ops-slack

Events: critical, fatal

Retries: 3 (10s delay)

Queue: persistent (survives restart)

Retry Queue

Failed webhook deliveries are queued and retried with exponential backoff.

```
$ aether notify status
```

NAME	TYPE	DELIVERED	PENDING	FAILED
ops-slack	webhook	142	0	0
pager-duty	webhook	38	1	0
email-ops	email	89	0	2

Retry queue: 1 pending (next: 8s)

Dead letter: 2 (manual review)

Multi-Cloud Cost Estimation

Compare your workload costs across providers before you deploy. No surprises on the invoice.

```
$ aether cost my-web-app --provider all
```

Cost Estimation for: my-web-app
Spec: 2 replicas, 500m CPU, 256Mi RAM, 10Gi storage

Provider	Instance	Monthly	Yearly
AWS	t3.micro	\$18.40	\$220.80
Azure	B1s	\$16.80	\$201.60
GCP	e2-micro	\$15.20	\$182.40
DigitalOcean	s-1vcpu-512mb	\$8.00	\$96.00
Linode	Nanode 1GB	\$10.00	\$120.00

Cheapest: GCP e2-micro (\$15.20/mo)
Current: Podman (local) - \$0.00/mo



Cost Breakdown

```
$ aether cost my-web-app --provider aws --detail
```

AWS Cost Breakdown (t3.micro x2):

Compute:	\$14.40/mo	(78%)
Storage:	\$2.00/mo	(11%)
Network:	\$1.50/mo	(8%)
Other:	\$0.50/mo	(3%)
Total:	\$18.40/mo	



Cost Optimization Tips

```
$ aether cost my-web-app --optimize
```

Optimization Suggestions:

- 1. Use spot/preemptible instances
Savings: ~60% (\$7.36/mo on AWS)
- 2. Right-size resources
CPU 500m → 250m (avg usage: 18%)
Savings: ~20% (\$3.68/mo)
- 3. Reserved pricing (1yr)
Savings: ~30% (\$5.52/mo)

Potential savings: up to \$16.56/mo

5
Cloud Providers

Real-time
Price Updates

60%
Max Spot Savings

JSON
Export Format

Set Up **Monitoring** Now

From zero to full observability in three commands.

1 Start the metrics server

Expose Prometheus metrics on port 9090 and configure your scrape target.

2 Import the Grafana dashboard

Export the pre-built 14-panel dashboard and import it into your Grafana instance.

3 Configure webhook alerts

Add Slack, PagerDuty, or email endpoints for critical and fatal events.

```
# Complete setup in 3 commands
$ aether metrics --port 9090 --daemon
$ aether metrics dashboard --export grafana > dashboard.json
$ aether notify add --name ops-slack --type webhook \
  --url https://hooks.slack.com/T0/B0/xxx \
  --events "warning,critical,fatal"
```



Further Reading

- Metrics reference: all 50+ exposed metrics
- Custom alerting rules with aether alert
- Long-term metric storage with remote write
- Multi-cluster federation setup



Integrations

- Prometheus + Grafana (native)
- Datadog (via StatsD bridge)
- OpenTelemetry (OTLP export)
- Slack, PagerDuty, Opsgenie webhooks

Observability Should Be Built In, Not Bolted On

Aether ships with everything you need — no sidecars, no agents, no external dependencies. Just start the metrics server and you are live.